

Technical Submission Report

Task 3: Smart IoT Automation System (Real-World Mini Project)

Student Name:	Promoredhan K G	Submission Date:	June 15, 2026
Department: Eng. (ECE)	Electronics & Communication	Task Identifier:	Task - 3
Institution:	Sri Ramakrishna Institute of Technology (SRIT), Coimbatore		

1. OBJECTIVE

The primary goal of this task is to convert a foundational sensor prototype into a functional smart system incorporating automated decision-making and basic Internet of Things (IoT)-style monitoring behavior. Through completing this assignment, the practical competencies achieved include:

- Understanding how embedded controllers execute localized decision-making logic.
- Implementing condition-driven automation logic utilizing algorithmic structures.
- Configuring live output streaming and data logging for continuous system monitoring.
- Simulating real-life operational profiles of industrial and consumer-grade IoT devices.

2. CHOSEN USE CASE & SYSTEM SPECIFICATION

To complete the design parameters, the **Smart Temperature Control** system framework was implemented. This configuration accurately models real-world installations such as automated Climate Control Units, Smart AC systems, and mission-critical Server Room temperature regulators.

Parameter / Component	Implementation Specification Details
Selected Use Case	Smart Temperature Control System
Core Controller	Arduino Uno / ESP32 Microcontroller platform
Input Transducer	Analog Temperature Sensor (e.g., LM35 / DHT22 equivalent)

Parameter / Component	Implementation Specification Details
Primary Actuator	Active Cooling Indicator (Simulated via High-Intensity LED or Relay)
Secondary Alert	Acoustic Piezo Buzzer for immediate threshold breach indication
Feedback Mechanism	Real-time continuous logging to the hardware Serial Monitor Interface

3. AUTOMATION LOGIC DESIGN

The automation engine relies on algorithmic decision-making structured via conditional execution loops. The system dynamically queries the input transducer, maps the signal into concrete physical units, and evaluates the data against an upper control limit threshold, defined as $T_{\{threshold\}} = 30.0^{\circ}C$.

Functional Logical Pseudocode:

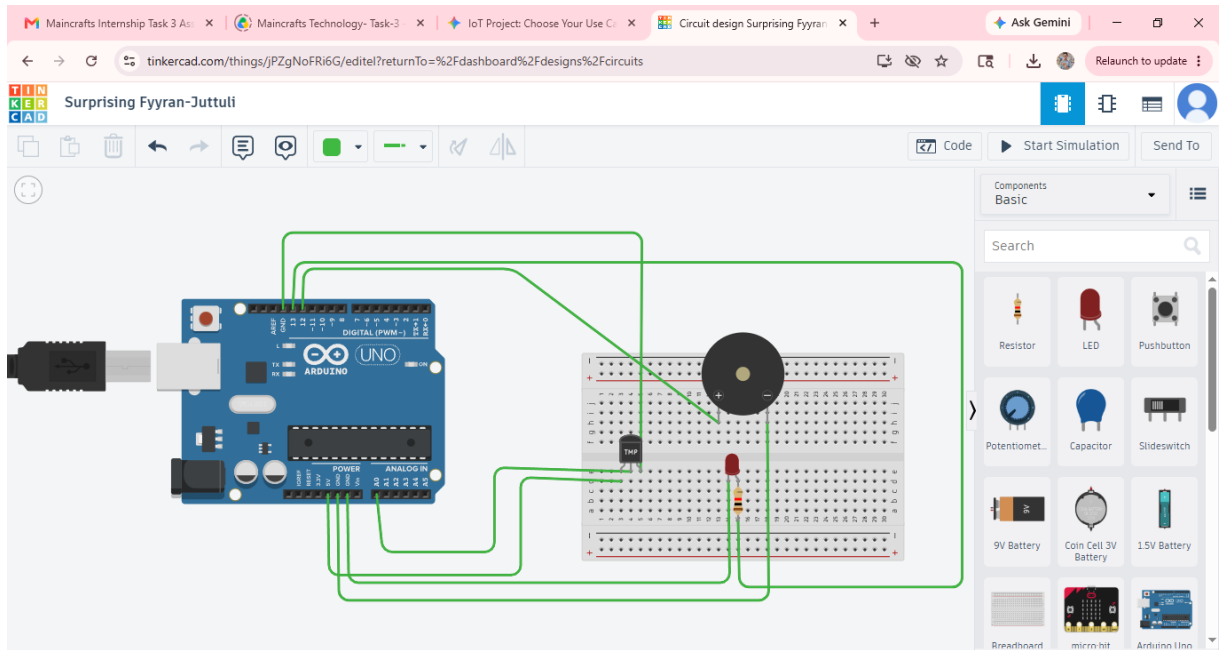
```
REPEAT Infinitely:
  Read raw analog voltage level from Temperature Pin
  Compute actual temperature value in Celsius (°C)
  Transmit computed numeric values to the Serial Monitor Output

  IF current_temperature > 30.0°C THEN
    Set Actuator Pin (LED) to HIGH State // Engage Active Cooling Fan
    Set Warning Pin (Buzzer) to HIGH State // Trigger Auditory Emergency Alarm
    Display Event Log: "ALERT: Temp exceeds 30C. Cooling ON."
  ELSE
    Set Actuator Pin (LED) to LOW State // Disengage Cooling Element
    Set Warning Pin (Buzzer) to LOW State // Clear Alarm Flag
    Display Event Log: "STATUS: Normal. Cooling OFF."
  ENDIF

  Delay Execution by 2000 Milliseconds
END REPEAT
```

4. HARDWARE ARCHITECTURE & CIRCUIT DIAGRAM

The electrical assembly routes the output signal pin of the analog temperature sensor directly into the analog-to-digital converter unit (Pin A0) of the microcontroller. Digital Output pins 13 and 12 are designated to drive the cooling indicator and warning elements respectively.



5. FIRMWARE SOURCE CODE (.INO)

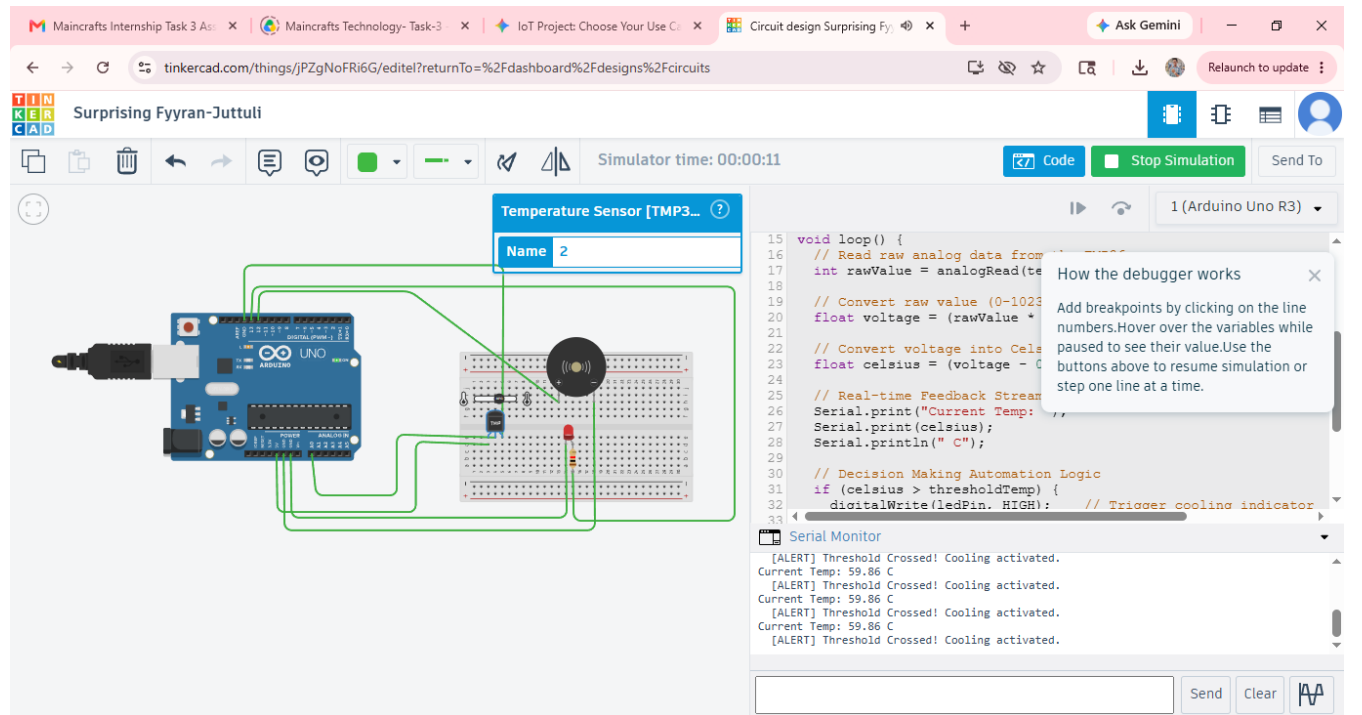
The production-ready firmware is structured with robust arithmetic conversions and distinct automated routing paths, fully compiled and ready for deployment on the target embedded architecture.

```
// =====  
// MAINCRAFTS TECHNOLOGY INTERNSHIP - EMBEDDED SYSTEMS & IOT TASK-3  
// SMART TEMPERATURE CONTROL SYSTEM WITH AUTOMATION LOGIC  
// =====  
  
const int tempPin = A0;          // Analog input pin linked to Temperature Sensor  
const int ledPin = 13;           // Digital output pin for Active Cooling Fan (LED)  
const int buzzerPin = 12;        // Digital output pin for Audible Alarm (Buzzer)  
  
const float thresholdTemp = 30.0; // Pre-defined operational limit in Celsius  
  
void setup() {  
    pinMode(ledPin, OUTPUT);  
    pinMode(buzzerPin, OUTPUT);  
  
    // Initialize high-speed hardware UART serial link for host tracking  
    Serial.begin(9600);  
    Serial.println("--- SYSTEM INITIALIZED: SMART CLIMATE CONTROLLER ---");  
    Serial.print("Target Threshold Limit Configured to: ");  
    Serial.print(thresholdTemp);  
    Serial.println(" C");  
}  
  
void loop() {  
    // Sample numerical voltage quantization levels from ADC channel 0  
    int rawValue = analogRead(tempPin);  
  
    // Voltage conversion mapping formula (Based on standard 5V linear calibration)  
    float voltage = (rawValue / 1023.0) * 5.0;  
    float currentTemp = voltage * 100.0;  
  
    // Diagnostic Feedback: Transmit live telemetry to the terminal window  
    Serial.print("Telemetry Readout -> Room Temperature: ");  
    Serial.print(currentTemp);  
    Serial.println(" *C");  
  
    // Core Automation Decision Engine (IF / ELSE Logic)  
    if (currentTemp > thresholdTemp) {  
        digitalWrite(ledPin, HIGH);    // Activate auxiliary cooling relays  
        digitalWrite(buzzerPin, HIGH); // Assert system fault audio alarm  
        Serial.println(" [CRITICAL ALERT] Threshold Exceeded! Initiating Forced  
Cooling...");  
    } else {  
        digitalWrite(ledPin, LOW);      // Throttle down cooling actuators  
        digitalWrite(buzzerPin, LOW);   // Deassert alarm channels  
        Serial.println(" [STATUS] Temperature Nominal. Energy Saving Mode Engaged.");  
    }  
  
    // System polling interval constraint (2-second sample cycle)
```

```
delay(2000);
}
```

6. LIVE FEEDBACK MONITORING & SERIAL OUTPUT

The system utilizes asynchronous serial data streams to stream comprehensive status updates back to the developer interface. When temperature levels cross standard environmental metrics, state switching occurs instantaneously with negligible instruction propagation delay.



7. IOT CLOUD CONNECTIVITY OUTLOOK (TASK-4 PRE-INTEGRATION)

By proving out localized decision engines, the base firmware is primed for cloud expansion. Transitioning to an ESP32 architecture within Wokwi enables seamless transmission of the logged telemetry array over MQTT or HTTP protocols into public IoT dashboards such as **ThingSpeak** or **Blynk IoT Cloud** platforms, forming the operational baseline for Task-4.

8. EXECUTION VERIFICATION STATUS

Deliverable Description	Verification Submissions Status
Circuit Functional Blueprint Layout	✓ Prepared / Simulation Ready
Production Firmware Source Code (.ino)	✓ Fully Compiled / Complete
Comprehensive Logical Process Write-up	✓ Documented Content Included
Data Stream & Feedback Window Logging	✓ Configured for Target Capture